COMP 322: Standard formulas

Computer Vision and Image Processing, Fall 2026

Hong Xu

This note fixes the notation for the formulas used so far: filters and convolution, sampling, Fourier transforms, edges, camera geometry and calibration, and the deep-learning and CNN material. When a later homework writes one of these, it should match the symbols here.

1 Notation

Symbol	Meaning
$I[m, n]$	Image value at row m, column n. Continuous form: $I(x, y)$.
$f[k, l]$	Filter, also called the kernel.
$h[m, n]$	Filter output.
$*$	Convolution: the kernel is flipped before the multiply-add.
$\star$	Correlation: the kernel is used as written.
σ	Standard deviation of a Gaussian.
f, $f_{\max}$, f_s	Frequency, highest frequency in a signal, sampling frequency.
$\mathbf{X}$, $\mathbf{x}$	World point and image point, homogeneous.
K, R, $\mathbf{t}$, M	Intrinsics, rotation, translation, and the 3×4 camera matrix.
W, $\mathbf{b}$, s	Weights, bias, and class scores (logits).

Indices m, n locate a pixel. Indices k, l locate a tap inside the kernel. A sum written $\sum_{k,l}$ runs over every tap of the kernel.

Two products show up under the same English word "convolution." In the filtering lectures, $*$ flips the kernel and $\star$ does not. In PyTorch, `Conv2d` does not flip the kernel, so it implements $\star$ and still calls the result a convolution. The CNN section below uses the PyTorch convention and says so.

2 Filters

A filter replaces each sample by a function of a local window. A *linear* filter is a weighted sum of that window. The same weights slide to every pixel, which is shift invariance.

2.1 Correlation and convolution

Two-dimensional correlation, the kernel as written:

$$h[m, n] = \sum_{k,l} f[k, l]\, I[m + k, n + l]. \tag{1}$$

Two-dimensional convolution flips the kernel:

$$h[m,n] = (I * f)[m,n] = \sum_{k,l} f[k,l]\, I[m-k, n-l]. \tag{2}$$

So $I * f$ equals correlation of I with f rotated by 180°. If f is symmetric under that rotation, the two operations agree. A Gaussian is symmetric. A Sobel kernel is not.

In SciPy, `correlate2d(I, f)` is (1) and `convolve2d(I, f)` is (2).

2.2 What a linear filter may do

Let $*$ stand for either product when the identity does not depend on the flip.

$$I * (f_1 + f_2) = I * f_1 + I * f_2, \qquad \text{linearity,} \tag{3}$$

$$(aI) * f = a\,(I * f), \qquad \text{scalars factor out,} \tag{4}$$

$$a * b = b * a, \qquad a * (b * c) = (a * b) * c. \tag{5}$$

Convolution also distributes over addition. The last two identities are for convolution only: correlation is not commutative and not associative, because each correlation flips only one argument. The identity kernel is a 1 at the center and 0 elsewhere. In one dimension, $e = [0,0,1,0,0]$ and $a * e = a$.

Shift invariance: shifting the image and then filtering is the same as filtering and then shifting. Near the border the window hangs off the image. The usual fills are: zeros (clip), wrap, copy the edge pixel, or reflect across the edge. Output sizes, with a kernel of odd width $K = 2r + 1$:

- `full`: pad with zeros, output is larger by r on each side.
- `same`: return the original height and width.
- `valid`: no padding, output shrinks by r on each side.

2.3 Kernels used in class

The box (moving average) replaces each pixel by the mean of its neighborhood. For a 3×3 window every weight is $1/9$, and the weights sum to 1, so a constant image is unchanged.

The Gaussian, continuous and separable:

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} \exp\!\left(-\frac{x^2+y^2}{2\sigma^2}\right) = G_\sigma(x)\, G_\sigma(y). \tag{6}$$

A discrete kernel is a sampled Gaussian, normalized so the taps sum to 1. The rule of thumb is a half-width of about 3σ, because the tails are then near zero. Convolution of two Gaussians is a Gaussian, and two passes with width σ equal one pass with width $\sigma\sqrt{2}$.

Sobel, horizontal and vertical derivatives. S_x responds to a vertical edge:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}. \tag{7}$$

The row or column $[1,2,1]$ is a small binomial smooth, a discrete stand-in for a Gaussian, applied along the edge.

Sharpening accentuates the difference from the local average. If e is the identity kernel and b is a box that sums to 1,

$$f_{\text{sharp}} = 2e - b, \tag{8}$$

which also sums to 1.

A median filter writes the middle value of the window after sorting. It removes salt-and-pepper noise without the blur of a mean. It is not a convolution: there are no fixed weights.

2.4 Separability and template matching

If $f[k,l] = u[k]\,v[l]$, a 2D convolution is two 1D convolutions. On an $M \times N$ image and a $P \times Q$ kernel the multiply-add counts are about $MNPQ$ for the 2D pass and $MN(P+Q)$ for the separable pass. The speedup is $PQ/(P+Q)$. A 9×9 kernel is about 4.5 times cheaper when it separates. The Gaussian separates.

Template matching slides a patch across the image and looks for a peak. Use correlation when the patch is not symmetric. Raw correlation grows with brightness. Subtract the mean of the image and of the patch first, so a bright region is not an automatic match:

$$I_0 = I - \overline{I}, \qquad f_0 = f - \overline{f}, \qquad h = I_0 \star f_0. \tag{9}$$

3 Sampling and aliasing

Subsampling by 2 keeps every other row and every other column. An image pyramid repeats that until one pixel remains. Without a low-pass filter first, high frequencies fold into lower ones. Two different signals then produce the same samples. That is aliasing.

Nyquist–Shannon: a signal whose highest frequency is $f_{\max}$ is determined by its samples when

$$f_s \geq 2 f_{\max}. \tag{10}$$

The Nyquist frequency is $f_s/2$. Frequencies above it are aliases of frequencies below it. Before keeping every other pixel, remove frequencies above half the new sampling rate. A Gaussian does that. The blur throws away detail, and that is preferable to aliasing. Downsampling by 2 in practice:

1. $I_{\text{blur}} = I * G_\sigma$ with a small Gaussian, for example 7×7 and $\sigma = 1$.
2. $I_{\text{small}} = I_{\text{blur}}[:: 2, :: 2]$.

A Gaussian pyramid is this operation repeated. A high-pass image is the original minus a low-pass image. A hybrid image adds the low frequencies of one picture to the high frequencies of another.

4 Fourier transforms

A periodic signal is a sum of sines and cosines. The square-wave example from lecture, with fundamental frequency f,

$$g(t) = \sin(2\pi f t) + \frac{1}{3}\sin(2\pi(3f)t) + \cdots \tag{11}$$

keeps the odd harmonics with amplitudes $1, 1/3, 1/5, \ldots$.

Euler's formula turns amplitude and phase into one complex weight:

$$e^{i\theta} = \cos\theta + i\sin\theta. \tag{12}$$

The analysis transforms:

$$G(f) = \int_{-\infty}^{\infty} g(t)\, e^{-i2\pi f t}\, \mathrm{d}t, \qquad\qquad G[k] = \sum_{n=0}^{N-1} g[n]\, e^{-i2\pi k n/N}. \tag{13}$$

The discrete sum is the DFT. With an image of width N, the frequency index runs over $k = -N/2, \ldots, N/2$, and the highest frequency that is not an alias is the Nyquist frequency $N/2$. The fast Fourier transform computes this in $O(N \log N)$ rather than $O(N^2)$.

Write $G[k] = |G[k]|\, e^{i\phi[k]}$. The amplitude $|G[k]|$ says how much of frequency k is present. The phase $\phi[k]$ says where that wave sits. A spatial shift leaves the amplitude unchanged and adds a term linear in k to the phase. For a real signal, $G[-k] = \overline{G[k]}$, so the displayed 2D spectrum is symmetric through the origin. The zero-frequency bin is the mean of the signal (the DC offset). Natural images are bright at low frequencies and fall off toward high frequencies. Most of what we recognize sits in the phase.

The convolution theorem is the reason the transform is worth computing:

$$\mathcal{F}\{I * f\} = \mathcal{F}\{I\} \cdot \mathcal{F}\{f\}, \tag{14}$$

a pointwise product of the transforms. The steps are: FFT of the image, FFT of the kernel, multiply, inverse FFT. Both arrays are padded to the same size, a power of two, before the transform. For a large kernel this is cheaper than sliding the kernel in space.

A box in space is a sinc in frequency, and the reverse:

$$\operatorname{sinc}(x) = \frac{\sin x}{x}. \tag{15}$$

The sharp cutoff of a box needs every frequency, which is why a box blur rings (the Gibbs effect) and a Gaussian blur does not. The transform of a Gaussian is a Gaussian whose variance is inverted: a wide blur in space is a narrow blob in frequency. Deconvolution divides by $\mathcal{F}\{f\}$. Where that transform is zero, or tiny compared with noise, the quotient blows up. A Gaussian is zero only at infinity, but in floating point the high-frequency bins are small enough that division is unstable. That is why deconvolution needs regularization even when the kernel is known.

JPEG applies a discrete cosine transform on 8×8 blocks. The coefficient $B(0, 0)$ is the DC term, the average of the block. Coefficients toward the lower right are higher frequencies and are quantized more coarsely.

5 Edges

An edge is a sharp change in intensity. Along a scanline the edge sits at an extremum of the derivative. Noise makes a raw derivative useless, so the image is smoothed first. Differentiation passes through convolution:

$$\frac{\mathrm{d}}{\mathrm{d}x}(I * G) = I * \frac{\mathrm{d}G}{\mathrm{d}x}. \tag{16}$$

One derivative-of-Gaussian kernel does both jobs. On pixels the smallest step is 1, and the centered difference estimates the derivative at x rather than halfway to the next pixel:

$$I_x[x] \approx \frac{I[x+1] - I[x-1]}{2}, \qquad I_x[x] \approx I[x+1] - I[x] \quad \text{(one-sided)}. \tag{17}$$

In two dimensions,

$$\|\nabla I\| = \sqrt{I_x^2 + I_y^2}, \qquad \theta = \operatorname{atan2}(I_y, I_x). \tag{18}$$

θ is the direction of steepest ascent, perpendicular to the edge. A large σ finds coarse edges and a small σ finds fine ones. Smoothing removes noise and also widens the ridge, so the peak is less sharply located.

Canny's detector, in order:

1. Filter with $\partial G_\sigma/\partial x$ and $\partial G_\sigma/\partial y$. Sobel is the fast integer version of the same idea.
2. Form the magnitude and the orientation θ.
3. Non-maximum suppression: along the gradient, keep a pixel only when it is strictly larger than both neighbors. Those neighbors are interpolated when θ is not along a pixel axis.
4. Hysteresis: pixels above a high threshold start an edge, pixels below a low threshold are dropped, and pixels in between are kept only when they connect to a strong edge.

6 Camera geometry

6.1 Planes and 2D maps

Scaling, rotation, and shear are linear. Translation is not, until the point is written in homogeneous coordinates. A Cartesian point (x, y) becomes $(x, y, 1)$, and (X, Y, Z) becomes $(X, Y, Z, 1)$. The same Cartesian point is any nonzero scale of that vector: (x, y, w) and $\lambda(x, y, w)$ are one point, recovered by

$$(x, y) = \left(\frac{x_h}{w}, \frac{y_h}{w}\right) \quad \text{when } w \neq 0. \tag{19}$$

A point with $w = 0$ is a direction, the image of a point at infinity. Parallel lines meet there.

Uniform scale, rotation by θ, and translation:

$$S = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix}, \qquad R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \qquad \begin{bmatrix} x' \\ y' \end{bmatrix} = R_\theta \begin{bmatrix} x \\ y \end{bmatrix}. \tag{20}$$

$R_\theta^{-1} = R_{-\theta} = R_\theta^{\mathsf{T}}$. An affine map is any composition of scale, rotation, shear, and translation. In homogeneous form it is one matrix

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \tag{21}$$

Affine maps send lines to lines and parallel lines to parallel lines. A projective map, or homography, is an invertible 3×3 matrix defined up to scale, so it has 8 degrees of freedom. Lines stay lines. Parallel lines need not stay parallel.

6.2 The pinhole

The camera at the origin looks along $+Z$. A world point (X, Y, Z) with $Z > 0$ hits the plane $Z = f$ at

$$u = f\frac{X}{Z}, \qquad v = f\frac{Y}{Z}. \tag{22}$$

f is the focal length. Length, area, and angle are lost. Straight lines survive. Parallel lines in the world meet at a vanishing point. Parallel lines on one plane meet on a vanishing line, for example the horizon.

The same map with a principal point (c_x, c_y), two focal lengths, and skew s is $\mathbf{x} = K[R \mid \mathbf{t}]\,\mathbf{X}$:

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad [R \mid \mathbf{t}] = \begin{bmatrix} R & \mathbf{t} \end{bmatrix}, \qquad M = K[R \mid \mathbf{t}]. \tag{23}$$

$\mathbf{X} = (X, Y, Z, 1)^{\mathsf{T}}$ and $\mathbf{x} = (u, v, 1)^{\mathsf{T}}$ are homogeneous, so the Cartesian pixel is $(x_1/x_3,\, x_2/x_3)$. K has 5 degrees of freedom. R and $\mathbf{t}$ have 6. The product M is 3×4 and defined up to scale, so a

general camera has 11 degrees of freedom. The lecture's $(\alpha, \beta, u_0, v_0)$ is this same K, with $f_x = \alpha$, $f_y = \beta$, and $(c_x, c_y) = (u_0, v_0)$.

Equation (22) is the special case $R = I$, $\mathbf{t} = \mathbf{0}$, $s = 0$, $c_x = c_y = 0$, and $f_x = f_y = f$.

Orthographic projection drops the division by Z (the camera is infinitely far, and rays are parallel). Weak perspective, or scaled orthographic, uses one depth for a whole nearby object: $u = (f/Z_0)X$ and $v = (f/Z_0)Y$.

7 Calibration

7.1 A line, as the warmup

Given points (x_i, y_i), the best line $y = mx + b$ in the least-squares sense minimizes

$$E = \sum_i (y_i - mx_i - b)^2 = \|A\mathbf{p} - \mathbf{y}\|^2, \tag{24}$$

with one row $(x_i, 1)$ of A per point and $\mathbf{p} = (m, b)^\mathsf{T}$. The normal equations and the closed form are

$$A^\mathsf{T} A\,\mathbf{p} = A^\mathsf{T}\mathbf{y}, \qquad \mathbf{p} = (A^\mathsf{T}A)^{-1}A^\mathsf{T}\mathbf{y}. \tag{25}$$

`numpy.linalg.lstsq` solves this. The same pattern, with a homogeneous right-hand side, is the camera estimate below.

7.2 The direct linear transform

Write $M = (m_{ij})$ and $\mathbf{x} = M\mathbf{X}$, so $u = x_1/x_3$ and $v = x_2/x_3$. Clearing the denominator gives two equations per correspondence $(X, Y, Z) \leftrightarrow (u, v)$:

$$\begin{aligned} 0 &= m_{11}X + m_{12}Y + m_{13}Z + m_{14} - u(m_{31}X + m_{32}Y + m_{33}Z + m_{34}), \\ 0 &= m_{21}X + m_{22}Y + m_{23}Z + m_{24} - v(m_{31}X + m_{32}Y + m_{33}Z + m_{34}). \end{aligned} \tag{26}$$

Stacking (26) for every point produces $A\mathbf{m} = \mathbf{0}$, where $\mathbf{m}$ is the 12 entries of M in row-major order. The zero vector satisfies this and is useless. The estimate is the unit vector $\mathbf{m}$ that minimizes $\|A\mathbf{m}\|$, which is the last right singular vector of A:

$$A = U\Sigma V^\mathsf{T}, \qquad \mathbf{m} = V_{:,12} \quad \text{(the last row of } \texttt{Vh} \text{ from } \texttt{numpy.linalg.svd}\text{)}. \tag{27}$$

Reshape $\mathbf{m}$ into the 3×4 matrix M.

The camera center $\mathbf{C} = (X, Y, Z, W)^\mathsf{T}$ is the world point that cannot be imaged: $M\mathbf{C} = \mathbf{0}$. It is the last right singular vector of M. Divide the first three entries by W to return to Cartesian coordinates.

The reprojection error of a candidate M is the root mean square distance between measured pixels (u_i, v_i) and the pixels obtained from $M\mathbf{X}_i$:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}((u_i - \hat{u}_i)^2 + (v_i - \hat{v}_i)^2)}. \tag{28}$$

To split $M = K[R \mid \mathbf{t}]$, factor the left 3×3 block as $A = KR$ (RQ decomposition), force the diagonal of K to be positive, and set $\mathbf{t} = K^{-1}\mathbf{m}_4$ with $\mathbf{m}_4$ the last column of M. Then $f_x = K_{11}$, $f_y = K_{22}$, $s = K_{12}$, $c_x = K_{13}$, and $c_y = K_{23}$.

8 Deep learning

8.1 Distances, scores, and losses

k-nearest neighbors labels a query by the majority label among the k closest training points. The two distances used in lecture, for vectors $\mathbf{a}$ and $\mathbf{b}$:

$$d_1(\mathbf{a},\mathbf{b}) = \sum_i |a_i - b_i|, \qquad d_2(\mathbf{a},\mathbf{b}) = \sqrt{\sum_i (a_i - b_i)^2}. \tag{29}$$

k and the choice of distance are hyperparameters. They are chosen on a validation set, or by cross-validation, and the test set is touched once at the end. $k = 1$ fits the training set perfectly and does not tell you about new data.

A linear classifier turns a feature vector $\mathbf{x}$ into one score per class:

$$\mathbf{s} = f(\mathbf{x}, W) = W\mathbf{x} + \mathbf{b}. \tag{30}$$

s_c is the logit of class c. The softmax turns logits into probabilities, and the loss is the negative log probability of the true class y:

$$p_c = \frac{e^{s_c}}{\sum_{j=1}^{C} e^{s_j}}, \qquad L_i = -\log p_{y_i}, \qquad L = \frac{1}{N}\sum_{i=1}^{N} L_i. \tag{31}$$

If every class is equally likely, $p_y = 1/C$ and $L_i = \log C$. For $C = 10$ that is about 2.3, which is why a CIFAR run starts near there. In PyTorch, `CrossEntropyLoss` takes the logits and applies (31) itself. Do not softmax first.

A regression target is a real number, or a vector of them. The losses from the assignment are the mean absolute error and the mean squared error,

$$L_1 = \frac{1}{N}\sum_i |\hat{y}_i - y_i|, \qquad L_2 = \frac{1}{N}\sum_i (\hat{y}_i - y_i)^2. \tag{32}$$

`L1Loss` and `MSELoss` average over every element of an (N, D) prediction. Part 1 of the PyTorch assignment is the case $D = 1$.

Binary classification with one logit uses

$$L = -\frac{1}{N}\sum_i \Big(y_i \log \sigma(s_i) + (1 - y_i)\log(1 - \sigma(s_i)) \Big), \quad \sigma(s) = \frac{1}{1 + e^{-s}}, \tag{33}$$

which is `BCEWithLogitsLoss`. The sigmoid stays inside the loss. Two classes can instead be two logits and (31). That is the form that also works for ten classes.

8.2 Partials and the gradient

A loss depends on many numbers at once: every weight, every bias, and every value computed on the way to the output. The ordinary derivative $\mathrm{d}f/\mathrm{d}x$ is the slope of a function of one variable,

$$\frac{\mathrm{d}f}{\mathrm{d}x} = \lim_{h \to 0} \frac{f(x + h) - f(x)}{h}. \tag{34}$$

A partial derivative is that same limit with every other argument held fixed. Training asks, for each parameter separately, how the loss moves when only that parameter is nudged, so the derivative that answers the question is a partial:

$$\frac{\partial L}{\partial w} = \lim_{h \to 0} \frac{L(\ldots, w + h, \ldots) - L(\ldots, w, \ldots)}{h}. \tag{35}$$

The gradient of L with respect to a vector or a matrix is the array of all of those partials, stored in the same shape as the variable:

$$(\nabla_{\mathbf{w}} L)_j = \frac{\partial L}{\partial w_j}, \qquad (\nabla_W L)_{ij} = \frac{\partial L}{\partial W_{ij}}. \tag{36}$$

In one dimension the derivative is the slope. In several dimensions the gradient is the vector of partial derivatives, one along each coordinate. The slope felt while walking in a unit direction $\mathbf{u}$ is the directional derivative $\nabla L \cdot \mathbf{u}$. That slope is most negative when $\mathbf{u}$ points along $-\nabla L$, and the steepest slope has size $\|\nabla L\|$. The update $W \leftarrow W - \eta \nabla_W L$ is a step in that downhill direction. One partial would move only the coordinate it belongs to.

The two symbols d and ∂ mark different questions, and mixing them drops terms.

- $\partial L/\partial z$ is local. It holds the other inputs of that one function fixed. On an influence diagram it is the label on a single edge.
- $\mathrm{d}L/\mathrm{d}x$ is total. It follows every path by which L depends on x. If x reaches the loss through intermediates z_1 and z_2,

  $$\frac{\mathrm{d}L}{\mathrm{d}x} = \frac{\partial L}{\partial z_1}\frac{\partial z_1}{\partial x} + \frac{\partial L}{\partial z_2}\frac{\partial z_2}{\partial x}. \tag{37}$$

  The lecture's influence diagram says the same thing: sum the products of partials over every path out of x. Keeping only one partial omits the other paths.
- A gate with one input, such as a sigmoid of one pre-activation, has an ordinary derivative. A parameter vector has a gradient. Gradient descent steps with the gradient. `loss.backward()` stores, in `tensor.grad`, the total derivative of the scalar loss with respect to that tensor, in the tensor's own shape. Each entry of that tensor is still a partial, which is why the formulas below keep the symbol ∂.

8.3 Names on the deep-learning slides

The CMU 11-785 lecture uses a few names that this note does not take as the default. The objects are the same.

Divergence. $\mathrm{Div}(Y, d)$ is the error on one training pair. Y is the network output, written $\hat{\mathbf{y}}$ or $\mathbf{y}^{(N)}$ here. d is the desired output, the target, written y or t here. The per-example loss is that divergence:

$$\ell(\hat{\mathbf{y}}, y) = \mathrm{Div}(Y, d). \tag{38}$$

The loss, also called the empirical risk, is the divergences gathered over the training set. The slides sum them. This note averages them,

$$L = \frac{1}{N} \sum_{i=1}^{N} \ell(\hat{\mathbf{y}}_i, y_i). \tag{39}$$

The gradient of a sum is the sum of the gradients, and the gradient of the average is that sum divided by N. "The actual gradient is the sum or the average of the per-example derivatives" is the slide's way of saying which of those two gatherings is in use.

Ordinary d *and partial* ∂. At the output the slides write $\mathrm{d}\,\mathrm{Div}\,/\mathrm{d}\,\mathbf{y}^{(N)}$. The ordinary d is appropriate there: the target is held fixed, and this is the derivative of the divergence with respect to the network output. Inside the net, an edge is a partial, because that gate has several inputs and the edge is one of them. The backward pass multiplies the local partials. Where several edges leave a node, (37) adds the contributions, and the sum is the total derivative.

The overdot. A dot over a symbol means the derivative of the divergence with respect to that symbol. So $\dot{\mathbf{y}}$ means $\mathrm{d}\,\mathrm{Div}\,/\mathrm{d}\mathbf{y}$.

Layer symbols. Layer k computes a pre-activation and then an activation. With $\mathbf{y}^{(0)}$ the input,

$$\mathbf{z}^{(k)} = W^{(k)}\mathbf{y}^{(k-1)} + \mathbf{b}^{(k)}, \qquad \mathbf{y}^{(k)} = f_k(\mathbf{z}^{(k)}). \tag{40}$$

The pair (z, a) in the next subsection is $(\mathbf{z}^{(k)}, \mathbf{y}^{(k)})$. The function f_N is the activation on the output layer.

Bias as a weight. The slides append a constant 1 to the layer input and give that extra input its own weight. That weight is the bias, so the formulas can be written as one matrix product. The partial of the loss with respect to that extra weight equals $\partial L/\partial b$. This note keeps $\mathbf{b}$ as its own vector.

A row rather than a column. The slides define a derivative as the factor that multiplies a perturbation of the input to produce the perturbation of the output. For a scalar L and a column $\mathbf{x}$ of length n,

$$\Delta L = \frac{\mathrm{d}L}{\mathrm{d}\mathbf{x}}\,\Delta\mathbf{x}, \tag{41}$$

so $\mathrm{d}L/\mathrm{d}\mathbf{x}$ is a row of length n: the transpose of the shape of $\mathbf{x}$. They call that row the gradient. This note and PyTorch store $\nabla_{\mathbf{x}}L$ as a column of the same shape as $\mathbf{x}$. The entries agree. The chain-rule product is written in the opposite order, which the next subsection shows both ways.

8.4 The neuron and the scalar chain rule

A unit computes an affine value and then a scalar nonlinearity ϕ:

$$z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b, \qquad a = \phi(z). \tag{42}$$

The sigmoid is $\sigma(z)$ above. The ReLU is $\phi(z) = \max(z, 0)$. A threshold unit is constant almost everywhere, so it has no useful derivative. Sigmoid and ReLU are differentiable almost everywhere, which is what makes gradient descent possible. At the kink $z = 0$ the ReLU derivative is undefined. A subgradient may take the value 0 there. The slides flag this: the activation derivative in the backward pass may be a subgradient. Stacking linear layers with no nonlinearity collapses to one linear layer.

For a nested scalar function the chain rule multiplies ordinary derivatives, outer function first:

$$\frac{\mathrm{d}L}{\mathrm{d}x} = \frac{\mathrm{d}L}{\mathrm{d}z}\frac{\mathrm{d}z}{\mathrm{d}x}. \tag{43}$$

When $z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b$ has several inputs, each weight sees a partial:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z}\,x_j, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}. \tag{44}$$

For the running example $p = wx + b$ and $L = (p - t)^2$, with the other of w and b held fixed,

$$\frac{\partial L}{\partial w} = 2(p - t)\,x, \qquad \frac{\partial L}{\partial b} = 2(p - t). \tag{45}$$

The same pattern on a small graph with two inputs: let $q = x + y$ and $f = qz$, with the lecture values $x = -2$, $y = 5$, $z = -4$. Then $q = 3$ and $f = -12$. The local factors are $\partial f/\partial z = q = 3$ and $\partial f/\partial q = z = -4$, and each input of the sum has local partial 1. Sending -4 back along both edges gives $\partial f/\partial x = \partial f/\partial y = -4$. That is (37) with a product along each edge and a copy of the upstream derivative down every branch.

8.5 Backpropagation with vectors and matrices

A layer takes a vector in and produces a vector out, so the local factor on an edge is a matrix of partials. Vectors in this note are columns.

The Jacobian of $\mathbf{y} = f(\mathbf{x})$, with $\mathbf{y} \in \mathbb{R}^m$ and $\mathbf{x} \in \mathbb{R}^n$, is the $m \times n$ matrix

$$\left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)_{ij} = \frac{\partial y_i}{\partial x_j}. \tag{46}$$

Rows run over outputs and columns run over inputs. For the affine map $\mathbf{z} = W\mathbf{x} + \mathbf{b}$ the Jacobian $\partial \mathbf{z}/\partial \mathbf{x}$ is exactly W, which is the special case stated in the lecture. An elementwise activation $\mathbf{y} = \phi(\mathbf{z})$ has a diagonal Jacobian: the ith diagonal entry is $\phi'(z_i)$, and the off-diagonal entries are zero because output i does not depend on pre-activation $j \neq i$. A vector-valued activation that mixes coordinates has a full Jacobian.

The chain rule still puts the outer derivative first. In the slides' row convention, with $\mathrm{d}L/\mathrm{d}\mathbf{z}$ a row,

$$\frac{\mathrm{d}L}{\mathrm{d}\mathbf{x}} = \frac{\mathrm{d}L}{\mathrm{d}\mathbf{z}}\,\frac{\partial \mathbf{z}}{\partial \mathbf{x}}. \tag{47}$$

The product is a row of length n. In the column convention used here and stored by PyTorch, that row is transposed, which reverses the order and transposes the Jacobian:

$$\nabla_{\mathbf{x}} L = \left(\frac{\partial \mathbf{z}}{\partial \mathbf{x}}\right)^{\mathsf{T}} \nabla_{\mathbf{z}} L. \tag{48}$$

(47) and (48) contain the same partials. Applying one formula with the other layout produces a shape that cannot multiply, which is the check the slides point out: the dimensions have to match.

Three local rules are enough to walk a network backward. Let $\mathbf{g} = \nabla_{\mathbf{z}} L$, the column of partials of the loss with respect to the pre-activation.

Elementwise activation $\mathbf{y} = \phi(\mathbf{z})$, with the incoming column $\nabla_{\mathbf{y}} L$ already known:

$$\nabla_{\mathbf{z}} L = \phi'(\mathbf{z}) \odot \nabla_{\mathbf{y}} L. \tag{49}$$

For a sigmoid, $\phi'(z) = \sigma(z)\big(1 - \sigma(z)\big)$. For a ReLU, $\phi'(z) = 1$ when $z > 0$ and $\phi'(z) = 0$ when $z < 0$.

Affine map $\mathbf{z} = W\mathbf{x} + \mathbf{b}$, with W of shape $m \times n$, $\mathbf{x}$ of length n, and $\mathbf{z}$ of length m:

$$\nabla_{\mathbf{x}} L = W^{\mathsf{T}}\mathbf{g}, \qquad \nabla_W L = \mathbf{g}\,\mathbf{x}^{\mathsf{T}}, \qquad \nabla_{\mathbf{b}} L = \mathbf{g}. \tag{50}$$

The first line is (48) with $\partial \mathbf{z}/\partial \mathbf{x} = W$. The second line is the scalar rule $\partial L/\partial W_{ij} = g_i x_j$ written as one outer product, so $\nabla_W L$ has the shape of W. The third line is the scalar rule for a bias, one copy per output coordinate: the bias is added, so its partial equals the partial of the pre-activation it is added to. In the slide convention the same three results hold after the row $\mathrm{d}L/\mathrm{d}\mathbf{z}$ is turned into the column $\mathbf{g}$.

One forward and one backward pass of a single training pair:

1. Forward: from $\mathbf{y}^{(0)} = \mathbf{x}$, store every $\mathbf{z}^{(k)}$ and $\mathbf{y}^{(k)}$. The derivatives need those stored values.
2. At the output, form $\nabla_{\mathbf{y}^{(N)}}\ell$. For squared error on a scalar output, $\ell = (\hat{y}-t)^2$ and this derivative is $2(\hat{y}-t)$. For softmax cross-entropy on logits $\mathbf{s}$, with $\mathbf{p} = \mathrm{softmax}(\mathbf{s})$ and true class y, it is the column $\mathbf{p} - \mathbf{e}_y$.
3. Through the activation, $\nabla_{\mathbf{z}^{(k)}}\ell = \phi'(\mathbf{z}^{(k)}) \odot \nabla_{\mathbf{y}^{(k)}}\ell$.
4. Through the affine map, apply (50). Keep $\nabla_W \ell$ and $\nabla_{\mathbf{b}}\ell$ for the update. Pass $\nabla_{\mathbf{y}^{(k-1)}}\ell$ into the layer below.
5. The training gradient is the sum of these per-example gradients, or their average, according to whether L sums or averages the divergences. A minibatch replaces the full sum by the sum over the batch.

A shape check with numbers: $\mathbf{x} = (1, 2)^\mathsf{T}$, $W = (3, -1)$, $b = 0.5$, and $\ell = (z-1)^2$ with the identity activation. Then $z = 1.5$ and $\ell = 0.25$, so $g = \partial\ell/\partial z = 1$. Equation (50) gives $\nabla_W \ell = (1,\, 2)$, $\nabla_b \ell = 1$, and $\nabla_{\mathbf{x}}\ell = (3,\, -1)^\mathsf{T}$.

8.6 Gradient descent

Full-batch descent on a loss $L(W)$ with learning rate η:

$$W \leftarrow W - \eta \nabla_W L(W). \tag{51}$$

A minibatch of B examples estimates that gradient. Stochastic gradient descent uses the estimate in the same update. With momentum, a velocity v accumulates gradients. One common form, with momentum coefficient ρ (often 0.9):

$$v \leftarrow \rho v + \nabla_W L, \qquad W \leftarrow W - \eta v. \tag{52}$$

`loss.backward()` *adds* into the stored gradient. The training step therefore zeros it before the backward pass. A typical loop is: zero the gradient, forward, loss, backward, step.

Regularization adds a penalty so the weights stay small, for example $L \leftarrow L + \lambda \|W\|_2^2$. The learning rate is a hyperparameter. A step schedule multiplies it by a constant at fixed epochs. A linear warmup raises it from a small value at the start so a large initial rate does not explode the loss.

9 Convolutional networks

A fully connected layer on an image flattens $H \times W \times C$ numbers and connects every input to every output. A convolution reuses one small kernel at every location.

One output channel, with a kernel w of spatial size $K \times K$ over C_{in} input channels, in the PyTorch convention (no flip):

$$z[y, x] = b + \sum_{c=1}^{C_{\text{in}}} \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} w[c, i, j]\; I[c,\, y+i,\, x+j]. \tag{53}$$

Each output number is a dot product between the kernel and a $K \times K \times C_{\text{in}}$ patch, plus one bias. C_{out} kernels produce C_{out} activation maps, stacked on the channel axis. A ReLU is then applied elementwise.

Spatial size. Input W, kernel K, padding P, stride S:

$$W' = \frac{W - K + 2P}{S} + 1. \tag{54}$$

The same formula applies to the height. With $P = 0$ and $S = 1$, this is $W - K + 1$. The padding that preserves size for odd K and stride 1 is $P = (K - 1)/2$. A max-pool window uses (54) with K the window and S the pool stride. A 2×2 pool with stride 2 halves each spatial side.

Parameter count of one convolution, including biases:

$$C_{\text{out}} \cdot C_{\text{in}} \cdot K \cdot K + C_{\text{out}}. \tag{55}$$

A fully connected layer from D_{in} inputs to D_{out} outputs costs $D_{\text{in}}D_{\text{out}} + D_{\text{out}}$. The convolution is smaller because of local connectivity (each output sees a $K \times K$ neighborhood) and weight sharing (the same w at every position).

The receptive field is the set of input pixels that can affect one output. For stride-1 layers of kernel size K, each new layer adds $K - 1$ to the field. After L such layers the field has side

$$1 + L(K - 1). \tag{56}$$

Pooling or a stride greater than 1 grows the field faster, which is why a network downsamples instead of stacking 3×3 layers until the field covers the image.

A classification ConvNet ends in (31): one logit per class, integer labels, and `CrossEntropyLoss`. The last layer is linear in the flattened feature maps. No sigmoid or softmax is attached to the module when that loss is used.